

企业业务 Agent 模块解析手册(v2.0)

从普通聊天机器人,到可控、可复盘、可转线流沉的业务 Agent

适用对象:产品经理、业务负责人、运营、商务、技术支持 示例业务:API 接入与分析 Agent
版本:Agent 模块解析增强版 / 内部学习稿

Part 0: 开篇钩子 · 为什么是 Agent,为什么是现在

过去我们做的产品,是把流程写死、让用户照着点:点 A → 填 B → 提交 C,路径固定。Agent 第一次让我们能做「给个目标,让它自己想办法办成」的产品。这不是又一个技术名词,而是产品形态本身变了——从"设计每一步操作"变成"设计一个会自己决策的员工"。

它和"普通调了 AI 的产品"差在哪?

以前 · 调一次 API 的产品	现在 · Agent 产品
流程是你写死的:点 A→填 B→提交 C,路径固定	**流程是它自己决定的**:同一目标可能走不同路径
AI 只做一件小事:翻译/总结/分类	**会动手**:查库、调接口、改文件,不只是"说"
不会用工具、没有记性:每次调用一锤子买卖	**有记性、能多步**:查了 A 才知道下一步该查 B
出错是确定的:同输入同输出,好测	**结果是概率的**:这正是强大之处,也是要"上护甲"的原因

为什么偏偏是现在能做了?

模型终于"听得懂指令、还会用工具"了。三年前模型只会续写文本,无法可靠输出"调用天气接口的指令";现在能稳定理解任务、输出结构化工具调用——2023 年之后才真正成熟的能力。

Part 1: 什么是 Agent + Agent vs Workflow

Agent 的四要素

智能体(Agent)是任何能通过**传感器(Sensors)**感知**环境(Environment)**、并自主地通过**执行器(Actuators)**采取行动(Action)以达成特定目标的实体。四要素中,自主性(Autonomy)是关键所在——不是被动响应刺激或执行预设指令,而是基于感知和内部状态**独立决策**。

举个例子:API 接入场景的 Agent,它的环境是"三方接入诉求、API 文档、日志系统、凭证平台",传感器是"用户输入、文档检索、日志查询",执行器是"追问、生成排查建议、标记任务状态",自主性体现在"根据用户回答动态判断下一步该追问什么、该调哪个 Skill"。

自主性是分水岭:Agent vs Workflow

这是 Agent 和 Workflow 的本质区别:

- **Workflow**(工作流)是预先定义的结构化编排——静态流程图,如费用报销审批:提交→部门审批→财务审批→打款,每一步、每个分支都写死。Workflow 是**让 AI 按部就班地执行指令**。
- **Agent**(智能体)是以 LLM 为"大脑"的目标导向自主系统——你告诉它目标("帮三方排查签名失败"),它自己决定先追问什么信息、再调哪个 Skill、要不要查文档,没有写死的 if-then 规则。Agent 是**赋予 AI 自由度去自主达成目标**。

金句:"Workflow 是让 AI 按部就班地执行指令,而 Agent 则是赋予 AI 自由度去自主达成目标。"

Agent 的核心价值 = 基于实时信息进行动态推理和决策。同一个"签名失败"问题,Agent 可能因为用户回答不同,走出完全不同的排查路径——这在固定 Workflow 里做不到。

LLM Agent 和传统 Agent 的对比

维度	传统智能体	LLM 驱动智能体
核心引擎	显式编程的逻辑系统	预训练模型的推理引擎
知识来源	工程师预定义规则/算法/知识库	海量非结构化数据间接学习、内化
处理指令	结构化、精确的命令	高层级、模糊的自然语言
工作模式	确定性、可预测	概率性、生成式
泛化/适应性	弱,局限于预设框架	强,涌现能力
开发范式	规则设计、算法编程、知识工程	模型训练、提示工程、微调

LLM Agent 的三大工作特征(以旅行助手为例):① **规划与推理**(把"去北京玩三天"分解为订票→订酒店→规划路线);② **工具使用**(识别信息缺口主动调工具,如查天气、查航班);③ **动态修正**(用户说"不想去故宫",把它当新约束重新规划)。

何时才值得上 Agent?

任务规则固定、不需多步推理时,传统软件 + 一次 LLM 调用就够;Agent 只在"任务多步、依赖判断、流程会变"时才划算。

比如:

- 不值得:把用户输入翻译成英文 → 直接调一次翻译 API 即可,不需要 Agent
- 值得:排查三方接入的签名失败问题 → 需要多轮追问(缺什么补什么)、动态判断(可能是参数顺序错、可能是编码问题、可能是时间戳过期),每个案子的排查路径都不同

Part 2: Agent 怎么工作 · Agent Loop

循环是什么

Agent 的本质不是某种产品形态,而是一个循环——“想 → 做 → 看 → 再决定”:

```
Perception(■■,■■ Observation)
↓
Thought(■■ = ■■■■■?)
↓
Action(■■■■■■■■)
↓
■■ State Change(■■■■)
↓
■■ Observation(■■■■)
↓
■■■■(■■ Thought)
```

金句:“Agent 不是一种产品形态,而是一种控制策略——就是这个循环本身。”

业务例子:签名失败排查走一遍循环

场景:三方在钉钉群里说“你们接口一直签名失败,帮忙看下”

第 1 轮循环:

- **Observation(感知):**“签名失败”
- **Thought(思考):**“要排查签名问题,需要知道:接口路径、appid、签名参数原文、错误码、请求时间”
- **Action(行动):**追问用户“请提供:接口路径、appid、签名参数(原始未编码)、返回的错误码、请求时间”

第 2 轮循环:

- **Observation(感知):**用户回复了日志截图,里面有 appid、接口路径、签名参数
- **Thought(思考):**“签名参数里 order_id 在 timestamp 前面,但我们文档要求按字母顺序排,这里顺序错了”
- **Action(行动):**输出排查建议:“签名参数排序错误。文档要求:按参数名字母顺序排列后再拼接。您当前顺序是 orderid→timestamp,应改为 timestamp→orderid。修正后重新生成签名。”

第 3 轮循环(如果用户说"改了还是不行"):

- **Observation**:"改了顺序还是失败"
- **Thought**:"可能是编码问题,追问编码方式"
- **Action**:"请确认:签名前的字符串用的是 UTF-8 编码吗?参数值是否做了 URL 编码?"

这就是

Agent

Loop——每一轮"看到什么→想出下一步→做了什么",动态决策,没有写死的分支。

生产实践:ReAct 主循环 + 按需 Plan

生产环境 90% 的场景用「ReAct 主循环 + 按需 Plan」:

- 简单任务(签名排查、文档问答)→ 零规划开销,ReAct 边想边做,灵活高效
- 复杂任务(新 API 接入要走 5-8 步)→ 先用 Plan 工具生成 todo list,再按 list 逐步执行

设计逻辑:能不规划就不规划——"**不是越自主越好**",把规划权交给
自己判断"这个任务需不需要我先列个计划"。

Agent

Part 3: 企业落地的关键认知 · 为什么收敛成混合架构

承上启下的钩子

前面讲了 Agent 和 Workflow 的本质区别——**自主性是分水岭**:Workflow 按部就班执行指令,Agent 被赋予自由度自主达成目标。

但如果你去看真实落地的企业业务 Agent,会发现一个有意思的现象:它们看起来又很像 **Workflow**——有明确的主流程、有 Schema 校验、有固定的安全边界、有人工审核卡点。

既然 Agent 和 Workflow 有本质区别,为什么企业落地后两者又长得像了?

控制权钟摆:从全规则 → 全自主 → 混合架构

这不是巧合,而是工程实践的必然收敛:

第一阶段 · 早期(全用规则):

- 传统软件全是规则和 if-then 分支,死板但可控。
- 痛点:新场景要重新写规则,改一个流程要发版,无法应对"用户的问法千奇百怪"。

第二阶段 · 中期(全交 LLM 自主):

- LLM 出现后,开发者兴奋地把决策权全交给模型:"你自己看着办,想调什么工具就调什么"。
- 结果:灵活但失控——模型会"自作聪明"地跳过安全检查、会把敏感凭证写进日志、会在没确认清楚的情况下就调用生产接口。
- 金句:"业务最怕的不是系统不够聪明,而是系统自作聪明。"

第三阶段 · 收敛(混合架构):

- 工程实践把控制权收回来,收敛到:「Workflow 负责边界,Agent 负责弹性」
- **Workflow**(确定性)规定:哪些步骤必须走、哪些权限不能越、哪些结果必须审核(如:凭证发放必须人工确认、appsecret 不能进对话、上线前必须走检查清单)
- **Agent**(不确定性)在允许范围内处理:理解用户千奇百怪地问法、动态判断该追问哪些信息、从文档里检索相关段落、生成人话的排查建议

这不是偏好,是行业试错后的收敛终点:能力可以一直长,控制权必须留在确定性工程手里。

一句话收束

企业业务 Agent 不是"更会聊天的机器人",而是一个承接高频业务流程的工作系统——LLM 只负责理解与表达增强,由 Workflow、Schema、Skill、Task State、Memory、Audit 和人工审核共同负责稳定、可控、可复盘。

判断标准不是"回答得像不像",而是"能不能按流程稳定推进"。

为什么选混合架构:业务特征 → 需求 → 模块

公司里的 API 接入、联调排查、上线检查、单量分析,不是开放闲聊,而是**高频重复、规则明确、流程顺序稳定、凭证敏感**的业务流程。这些特征决定了你需要什么:

业务特征	Agent 需要什么	对应模块
流程顺序稳定	不要每次重想流程	Workflow
字段规则明确	判断可校验	Schema + Validator
问题高度重复	复用专家经验	Skill + 知识库
任务多轮推进	知道走到哪一步	Task State
凭证上线敏感	审计与人工确认	Audit + 人工审核

一句话:LLM 负责理解表达,流程、校验、状态、审计和人工确认负责稳定可控。

(Part 4-8 继续...)

Part 4: Agent 演进 · 怎么长成今天这样

在进入具体模块前,先看 Agent 是怎么一步步演进到今天的混合架构形态。这里有两条互补的演进叙事——一条看**自主度高低**(选型粗看法),一条看**能力补齐顺序**(被什么痛点逼出来)。

四阶段演进 · 自主度视角

从"会调工具"到"能自己决策并沉淀经验",Agent 经历了四个阶段:

阶段	特征	适用场景
早期 Agent	聊天 + 工具调用	短链路任务(查天气、翻译)
工作流 Agent	流程编排	企业稳定流程(报销审批)
自主 Agent	规划 + 执行	开放复杂任务(写报告、做调研)
自进化 Agent	Skill + Memory,沉淀经验资产	需要组织知识沉淀的场景

核心启发:对企业业务,不是越自主越好,而是要把模型能力放进可控流程里。第二阶段"工作流 Agent"恰恰是大量企业业务的最优解——流程稳定、边界清晰,LLM 只在理解表达环节发挥作用。

九阶段能力演进 · 补齐顺序

从另一个视角看,Agent 是一步步补齐能力模块,每一步都在解决上一代的痛点:

- 1. **LLM(地基):**能理解和生成,但无记性、知识静态、不会动手
- 2. **记忆 Memory:**补"每次调用都失忆"→ 短期上下文 + 长期持久化
- 3. **RAG:**补"不懂私有业务、还会编"→ 先检索再回答,没依据明说未找到
- 4. **Function Call & MCP:**补"只会说不会做"→ LLM
输出调用指令、代码执行;**决策与执行分离是安全可控的根**
- 5. **Agent Loop / ReAct:**补"多步任务谁来串"→ 想→做→看→再决定
- 6. **Skill:**补"工具一多就选不准"→ 渐进式加载 SOP + 工具,按手册操作而非临场猜
- 7. **Multi-Agent:**补"单个脑子带不动"→ 专家协作;成本翻倍,不到必要不拆
- 8. **Harness:**补"上线就崩"→
容错、限流、工具治理、人工审核、可观测;**从「能跑」到「能用」的分水岭**
- 9. **Claw(形态):**部署形态演进 → Agent 跑到你电脑上,直接动手并验证

归纳为三层:

- 第一层 · 知道什么(知识):②记忆 ③RAG
- 第二层 · 能做什么(能力):④Function Call & MCP ⑥Skill
- 第三层 · 怎么做得好(质量):⑤Agent Loop ⑦Multi-Agent ⑧Harness

金句:"形未变、神已变"——经典模块框架依旧(规划、记忆、工具、RAG),但每个模块的实现范式都被内核重构。底层思想是"通过工程化手段构建确定性,以承载模型不确定性"。

两条演进叙事的关系

- **四阶段**(自主度高低)是"选型粗看法":你的业务流程稳定吗?需要多大的自主度?
- **九阶段**(能力补齐)是"演进实况":每个能力是被什么痛点逼出来的,先后顺序是什么?

两者不矛盾,轴不同。本手册聚焦的"企业业务 Agent"通常落在第二阶段(工作流 Agent)或第二第三之间,能力上需要 ②-⑥ 这几个模块,⑦Multi-Agent 按需,⑧Harness 是上线红线。

Part 5: 各模块解析 · 混合架构怎么搭

这部分是手册核心,理解成一套业务 Agent 的标准工作分工——谁接任务、谁判断、谁追问、谁调用能力、谁保存状态、谁沉淀经验、谁管风险。

5.0 主链路全景

业务主流程:用户输入 → Session Router(会话路由)→ 需求转化 → Schema 校验 → 调用 Skill → 输出确认

LLM 增强层:LLM 作为"增强层"挂在多个节点上——识别意图(Session Router)、抽取字段(需求转化)、追问润色(Clarification Policy)、输出润色(结果合成),但LLM 是增强点,不是唯一决策者。

支撑与边界四件套:

- **Task State**:记住推进到哪一步(需求确认中、资料待补充、凭证待生成、沙箱联调中...)
- **Memory**:沉淀识别错误、用户纠正、真实联调案例,把个体经验变成组织能力
- **Audit**:记录每步操作(用户输入摘要、识别成什么、命中哪个 Skill、是否触发安全边界),但不记敏感信息
- **人工审核**:高风险操作的最后一道闸门(凭证发放、上线、改配置)

Workflow 归位说明:

Workflow 是贯穿全链路的骨架,定义「哪些步骤必须走、哪些权限不能越、哪些结果必须审核」,不单列为模块编号,而是体现在主链路的流程顺序里。比如:"新 API 接入"的 Workflow 规定必须先确认资料齐全、再生成凭证、最后人工审核上线,这三步顺序不能乱、不能跳。

LLM 放在哪里:适合做什么 vs 不应独自做什么

位置	LLM 适合做什么	不应该 LLM 独自做什么
Session Router	判断这句话像闲聊、补充信息还是新任务	不能因为模型觉得像业务就强行写入任务
需求识别与转化	把自然语言抽取成任务类型、字段、问题类型	不能绕过 Schema 自行认定可执行

追问策略	把阻塞项问得清楚、少问废话	不能为补全信息索要敏感凭证
知识问答	结合检索片段生成容易理解的回答	不能在无依据时编造接口规则
输出润色	把排查建议、分析结论写得更像人话	不能替代人工审批和生产操作

产品经理对外口径:

"我们不是让大模型直接接管业务,而是让大模型嵌入到可控流程里:它负责听懂、抽取、解释和表达;能不能执行、是否要追问、是否涉及安全边界,由 Schema、规则、权限和人工审核共同决定。"

5.1 主链路模块(按数据流顺序)

下面逐一解析主链路上的 8 个模块,按"用户输入→最终输出"的数据流顺序讲。

① 入口层:让人能把任务交给 Agent

是什么:用户和 Agent 接触的地方——WebUI(产品/商务/技术支持用)、CLI(技术/内测)、未来的本地接口或服务接口。

价值不是炫技,而是把业务需求自然地接进 Agent。例:商务在钉钉群里输入"某三方要接 API,问我们要 appid 和 appsecret",这句话能被 Agent 接住。

② Session Router(会话分诊台):先分清这句话是什么

是什么:判断这句话是闲聊、新任务、补充上一任务的信息,还是修改上一任务。

解决的痛点:避免把"好的""这个不对""换成昨天的数据"都强行当成新的 API 接入问题。

口诀:入口层负责接住人,Session Router
负责先分清话;一个让任务进得来,一个避免任务进错门。

产品经理要抓住的点:入口层负责让人,Session Router
负责分清话——一个让任务进得来,一个避免任务进错门。

③ 需求识别与转化:把自然语言变成结构化任务

是什么:把自然语言变成结构化业务任务。

例子:"对方说签名一直失败,帮忙看下" → 识别为:requesttype = signaturedebug →
问题类型 = 签名/鉴权排查 → 需要补充 =
接口路径、appid、错误码、请求时间、签名原始参数 → 安全提醒 = 不要提供 appsecret

LLM 适合:理解表达、识别意图、抽取参数、判断归类 **但只帮助理解,不负责最终放行** → 下一步交给 Schema 校验

④ Schema Builder + Validator:判断这个任务能不能继续

是什么:Schema 是业务任务单模板,Validator 是检查任务单是否合格的检查员。

校验结果三类:

- **executable:**可继续执行
- **needs_clarification:**需要追问(缺必填字段、表达模糊)
- **blocked:**触发规则或安全边界,不能继续(例:用户发了 appsecret 或要求 Agent 生成 appsecret → 进入 security_flags 而非继续对话)

金句:"LLM 可以帮你填表,但不能自己决定表是否合格;表是否合格,要靠 Schema 和业务规则校验。"

核心理解:LLM 可以从对话里抽取出 appid、接口路径、错误码,但不能自己认定"信息齐全了,可以执行了"——必须过 Schema 校验这一关,检查必填字段是否都有、是否触发安全边界(如:用户提供了 appsecret、要求生成凭证但没权限)。

⑤ Clarification Policy(追问策略):缺什么只问什么

是什么:追问策略——"缺什么就只问什么"。

痛点:很多 AI 助手不好用是因为一次问一堆问题,让业务人员烦,还把敏感信息带进对话。

追问四类:

- **缺失:**必须字段没有(如:排查签名但没提供 appid)
- **模糊:**表达不清楚(如:"昨天"是具体哪个时间段?)
- **冲突:**用户说法和规则冲突(如:要求续租但该号码在黑名单里)
- **非法:**触发权限或安全边界(如:非管理员要求查看 appsecret)

原则:可以让 LLM 做话术润色,但**问哪些字段由 Schema 和规则决定**。例:Schema 规定"signature_debug 任务缺 appid 就问、缺 appsecret 不问反而要提醒风险",LLM 只负责把"缺 appid"这个判断结果润色成"请提供您的 appid,我帮您排查"。

⑥ Skill Router + SkillLoader:把任务交给哪个专家

Skill Router 是什么:像派单系统——新接入→API 接入流程 Skill,字段问题→API 文档问答 Skill,签名失败→鉴权与签名排查 Skill,回调异常→回调排查 Skill,续租失败→续租规则 Skill,单量分析→单量分析 Skill。

SkillLoader 是架构增强点:不是一开始把所有 Skill 全塞给模型(会爆 context、模型也选不准),而是**分阶段加载**:

- **Advertise**(先知道有哪些 Skill):给模型一份 Skill 目录,每个 Skill 一句话介绍
- **Load**(命中后读取完整 Skill):模型判断要用"鉴权排查 Skill"后,再把这个 Skill 的完整 SOP 加载进来
- **Read**(需要时读取参考资料):Skill 执行中需要查 API 文档,再去检索相关段落
- **Run**(必要时运行脚本或工具):Skill 里定义了查日志的脚本,执行它

类比:Skill 是业务专家能力包(一个 Skill = 一套 SOP + 配套工具 + 参考资料),Skill Router 是把问题交给哪个专家,SkillLoader 是"按需翻手册"而非"把所有手册都背下来"。

⑦ 知识库与文档检索:回答要有依据

是什么:为了避免模型瞎编。API 业务资料包括 API 文档、FAQ、SDK 文档、接入流程、错误码说明、续租规则、货架/号池规则。

原则:回答应先检索文档,再基于文档回答,而不是让模型凭经验猜;**无命中文档依据时应明确说明未找到。**

解决:回答可追溯、口径一致。例:用户问"某字段什么意思",Agent 不是直接猜,而是先查 API 文档,找到字段说明后引用原文回答,并告诉用户"依据:API 文档 v2.3,第 5.2 节"。

⑧ Tool Governance(工具治理):工具能做事,但必须受控

是什么:未来接 BI 查询、日志查询、后台配置、凭证系统、订单系统等真实系统后,工具不再只是回答问题,而是能影响真实业务结果,必须受控。

要管理:

- 这个工具能不能调用?(权限白名单)
- 谁有权限调用?(角色/用户级权限)
- 是否只允许 dry-run?(一期可以只做查询、不做写入)
- 是否需要人工确认?(凭证生成、上线、改配置必须人审)
- 是否记录审计日志?(谁在什么时间调了什么工具、返回了什么)

一期策略:可以很轻量——只做样例脚本、dry-run、权限说明,不接真实后台;**接真实系统前再加强治理。**

边界提醒:越接近真实后台、真实凭证、真实数据,越需要权限、审计和人工确认。

5.2 支撑与边界模块

前面 8 个模块构成主链路,下面 4 个模块是"支撑与边界"——让 Agent 能多轮推进、能沉淀经验、能复盘风险、能守住最后防线。

⑨ Task State:让 Agent 知道任务推进到哪一步了

是什么:企业业务 Agent 和普通聊天机器人最大的区别之一。要记住任务状态:

- 需求确认中
- 资料待补充
- 凭证待生成
- 接入包待发放
- 沙箱联调中
- 上线前检查中
- 已上线
- 上线后监测中
- 暂停/阻塞

解决:多轮推进,而不是每轮从零开始。

例子:没有 Task State,用户说"资料补齐了,下一步呢",Agent 不知道之前走到哪一步,只能重新问一遍;有了 Task State,Agent 知道"上次卡在资料待补充,现在用户说齐了,下一步是生成凭证"。

⑩ Memory:把个体经验逐步变成组织能力

是什么:Memory 不是简单记住聊天记录,而是记录:

- 识别错误(Agent 把"续租失败"误判成"订单回调问题",用户纠正后记下来)
- 用户纠正(用户说"不是这个意思,我是想问...",记录纠正前后对比)
- 真实联调案例(一个完整的签名排查案例,从问题描述到最终解决)
- 重复问题(某个问题被问了 10 次,说明 FAQ 或文档不够清楚)
- 高频 FAQ(TOP 10 问题)
- Skill 优化建议(某个 Skill 的排查步骤有遗漏,用户反馈后形成优化建议)

解决:"把个体经验逐步变成组织能力"。

例子:比如 Agent 第一次遇到"续租失败"时走了弯路,误判成订单问题,用户纠正后记进 Memory;下次再遇到类似问法,Memory 告诉 Agent "上次这种情况是续租问题,不是订单问题",识别准确率就提高了。后续人工审核时,把这条纠错补充进续租 Skill 的 few-shot 示例,或者更新测试集。

Runtime Logs / Audit:让每次判断可复盘

是什么:企业 Agent 的安全底座。记录:

- 用户输入摘要(不记全文,避免敏感信息)
- 识别成什么任务
- Schema 校验结果(executable / needs_clarification / blocked)

- 缺了哪些字段
- 命中哪个 Skill
- 是否调用了 LLM(调了几次、token 数)
- 是否触发安全边界(security_flags)
- 最终输出摘要

但不能记录敏感信息:appsecret、真实密钥、完整敏感凭证。

解决:出了问题能复盘,有风险能审计,同时不把敏感信息沉淀到普通日志里。

例子:三方投诉说"你们给错了凭证",审计日志能回溯:这个任务是什么时间进来的、识别成了什么、Schema 校验结果是什么、最终是人工审核通过还是 Agent 自动发的——如果是自动发的,说明流程有漏洞(本该拦住的没拦住),而不是甩锅说"模型出错了"。Agent

人工审核发布:最后一道边界

是什么:最后一道边界。Agent 可以生成建议、整理清单、输出草稿、给排查步骤、生成 Skill 优化建议,但不能自动:

- 生成 appsecret
- 发放凭证
- 改后台配置
- 正式上线
- 自动更新生产 Skill

这层高风险操作必须人工确认。解决:企业 Agent 可控落地而非失控自动化。

金句:"Agent 是帮人推进流程,不是绕过人和权限系统。"

对内讲解必须讲清:Agent 的价值是把重复的信息整理、判断前置、排查建议工作做了,最终决策权和高风险操作的执行权还在人手里——这不是因为不信任 AI,而是因为企业流程本身就有"双人复核""审批流""权限分级"这些控制点,Agent 要嵌入到这些控制点里,而不是绕过它们。

5.3 一句话速记

Workflow 管流程,Schema 管判断,Clarification Policy 管追问,Skill 管能力,Task State 管推进,Memory 管进化,Audit 和人工审核管风险,LLM 负责理解和表达增强。

简版收束:

Task State 管推进,Memory 管进化,Audit 管风险,人工审核管最终放行。

Part 6: 如何落地 · PM 方法论

前面讲了混合架构的 12 个模块,但产品经理拿到一个新业务场景时,该从哪里开始?哪些模块一期必须做,哪些可以后续增强?验收标准怎么定?

一期范围:哪些必须做,哪些后续增强

这些模块可以作为企业业务 Agent 的标准底座,但第一次做时,不需要一开始就把所有模块都做成最完整版——总原则:核心链路先跑通,检索、日志、Memory 先轻量,真实工具和平台化能力后续再接。

模块	一期建议	原因
入口层	必须有	用户需要一个发起任务的位置
Session Router	必须有轻量版	避免闲聊、补充信息、新任务混在一起
需求识别与转化	必须有,可接 LLM	自然语言必须转成业务任务
Schema + Validator	必须有	判断能不能继续执行,不能只靠模型感觉
追问策略	必须有	只问阻塞项,减少用户负担
Skill Router + Skill	必须有	不同问题要走不同业务能力
知识库检索	一期轻量做	先用文档摘要和关键词检索,不必一开始做复杂向量库
Task State	必须有轻量版	支持多轮推进和草稿修改
Memory	一期轻量做	先记录误判、纠错、重复问题
Tool Governance	接真实系统前加强	涉及后台、BI、凭证时必须有权限和审计
Audit + 人工审核	必须有	这是企业场景的安全边界

落地七步

不要从"我要做一个万能 Agent"开始,而是先选一个明确业务闭环,拆成 Workflow、Schema、Skill、State 和验收场景:

- 1. **第一步:**选一个业务闭环,不做万能入口
- 2. **第二步:**写清 Workflow,也就是任务从发起到结束的主流程
- 3. **第三步:**定义 Schema,只保留会影响判断、追问、阻断的字段
- 4. **第四步:**整理一期 Skill,先覆盖高频 SOP、文档问答、排查、分析
- 5. **第五步:**接入 LLM 做理解和表达增强,但保留规则兜底和安全边界
- 6. **第六步:**做验收集,用真实问题压测识别、追问、路由和安全
- 7. **第七步:**把误判、纠错、重复问题进入 Memory,再由人工审核更新 Skill

扩展时机:当一个业务 Agent 能稳定处理核心场景,且日志、Memory、Skill 更新机制跑通后,再抽象成通用底座;此时新业务只需替换 Workflow、Schema、Skill、知识库和验收集,不用重新发明整个 Agent 架构。

选第一期场景的四条判断标准

不是所有业务都适合一期就上 Agent,先挑最容易出效果的:

- 1. **高频:**这个问题每天或每周反复出现
- 2. **规则明确:**不是完全靠人情判断,而是有文档、SOP、字段、审批边界
- 3. **可人工确认:**Agent 先出草稿或建议,人来最终确认
- 4. **能做验收:**可以拿真实问题或历史案例测试 Agent 是否答对

80 分验收口径

业务 Agent 的验收不要只看"回答好不好看",要看是否稳定走完正确链路,尤其是需求识别、追问、Skill 路由、安全边界和人工确认。

80 分验收口径:"初版不追求'无所不答',而是追求在核心场景里达到 80 分:识别稳定、追问克制、输出有依据、安全边界清楚。"

验收场景(API 接入示例)

验收场景	应该看到的结果
三方说"给一下 appid 和 appsecret"	识别为新接入或凭证相关请求;先检查资料;不生成、不复述 appsecret
三方说"签名失败"	追问接口路径、appid、签名参数、错误码、日志时间;提醒不要发 appsecret
三方说"没有收到回调"	追问回调地址、订单号、触发动作、服务端日志,并进入回调排查 Skill
三方问"某字段什么意思"	命中文档问答或参数解释 Skill,并说明依据来源

三方问"续租为什么失败"	按续租规则排查投诉、黑名单、时间冲突、未开始续租单等原因
内部问"某渠道近 7 天单量"	确认渠道、时间、游戏范围、指标口径,输出趋势、异常点和建议

产品经理对外口径(再次强调)

当业务同学问"这个 Agent 会不会自己乱来",用这段话统一口径:

"我们不是让大模型直接接管业务,而是让大模型嵌入到可控流程里:它负责听懂、抽取、解释和表达;能不能执行、是否要追问、是否涉及安全边界,由 Schema、规则、权限和人工审核共同决定。"

Part 7: API 接入与分析 Agent · 实战分析

前面讲了架构和方法论,这部分用 API 接入与分析 Agent 作为端到端示例,把 12 个模块串起来,看一个真实业务场景是怎么跑的。

为什么 API 接入适合第一批业务 Agent

API 接入场景天然适合第一批业务 Agent:

- 固定流程:新接入要走资料确认→凭证准备→上线检查,步骤明确
- 固定材料:appid、appsecret、接口文档、回调地址,字段固定
- 固定文档:API 文档、FAQ、SDK 文档、错误码说明
- 固定排查套路:签名失败看参数顺序和编码、回调异常看地址和触发条件、续租失败看黑名单和冲突
- 明确安全边界:appsecret 不能进对话、凭证发放必须人审、上线前必须检查清单

三方常见问题大量重复(签名失败、回调异常、某字段什么意思、续租规则),内部同学也需要持续做单量整理和上线后监控。

三条任务链:共用底座,各跑各的流程

API 接入与分析 Agent 不是一条线走到底,而是三条任务链分别跑流程,但共用底座(Schema / Skill / Task State / Memory / Audit):

1. 新 API 接入:需求确认 → 资料补齐 → 凭证准备 → 上线检查
2. 联调问题:识别问题 → 补齐日志 → 排查建议 → 沉淀案例
3. 单量分析:确认口径 → 趋势分析 → 异常判断 → 输出周报

安全边界(红线):

- appsecret 不进模型上下文
- appsecret 不在对话中复述
- appsecret 不写入普通日志
- 查看/重置/发放 appsecret 必须走有权限和审计的安全通道

端到端案例:三方说"签名失败"

用一个真实问题走一遍 8 个阶段,看 Agent 做什么、产品经理要关注什么:

阶段	Agent 做什么	产品经理要关注什么
入口层	接收"签名失败"这句话	入口是否足够方便,是否适合内部同学使用
会话路由	判断这是新问题还是延续上次联调	避免把无关上下文混进来
需求识别	识别为 `signature_debug`	是否能识别口语化表达
Schema 校验	发现缺接口路径、appid、签名原始参数、错误码、日志时间	缺的是不是阻塞项
追问策略	只追问上述关键字段,并提醒不要提供 appsecret	追问是否简洁、安全
Skill 路由	命中鉴权与签名排查 Skill	排查步骤是否来自 SOP/文档
输出结果	给出排查建议和下一步动作	是否能让技术支持直接拿去用
Memory/Audit	记录本次问题类型和安全提醒	是否能沉淀为案例,是否保护敏感信息

关键安全口径(再次强调):

appsecret 不进模型上下文、不在对话中复述、不写入普通日志;查看、重置、发放必须走有权限和审计的安全通道。

代码参考:从开源模板学设计

前面讲的是架构和模块,但产品经理可能还想知道"有没有现成的代码参考"?这里从开源项目 awesome-llm-apps 里挑几个和企业 Agent 模块对应的模板,提炼可直接抄的设计思路。

trust-gated 信任闸门(对应 Tool Governance + 人工审核)

模板:trustgatedagent_team

设计思路:多 agent 流水线(Researcher→Analyst→Writer),每个 agent 参与前必须先通过信任打分(0-100,金/银/铜分级),不达标直接被挡在门外;每步操作写进 SHA-256 哈希链审计日志,任一条被篡改后面全部对不上。

对应到企业 Agent:"参与前先过闸"的 gating 模式 = 凭证发放/上线/改配置前人工审核"达标才放行"闸门。哈希链审计日志 = Audit 模块的"可复盘、防篡改"要求。

用户版本要叠加的:敏感信息不进日志(appsecret 不落盘)——这是该模板未强调、但企业场景更严的地方。

deterministic-picker 思路(对应 Schema + Validator)

模板:ragfailediagnostics_clinic

设计思路:预定义 12 种失败模式(P01 Query Ambiguity、P02 Missing Context...),在 System Prompt 里硬写死"只能从这 12 个枚举里选,不许发明新 id",把判断交给代码而非 LLM。

对应到企业 Agent:Schema 需求识别的"LLM 帮你填表,代码决定合不合格"——LLM 可以从对话里抽取任务类型,但最终"这个任务类型合不合法、是不是在允许的枚举里",由代码校验,不让 LLM 自由发挥。

统一抄法三问

看任何开源模板时,问自己三个问题:

- 1. 把"判断"交给 LLM 还是代码? → 核心判断(能不能执行、是否触发安全边界)交给代码,表达和抽取交给 LLM
- 2. 兜底怎么处理? → 没命中就明说"未找到依据",而不是让 LLM 编
- 3. 沉淀了什么可复盘记录? → 每步操作、判断结果、是否触发边界,都要进 Audit

API 业务一期 Agent 能力范围

第一期不追求"无所不答",先把核心场景跑通:

- 接住新接入诉求,判断资料是否齐全,输出接入准备清单
- 回答 API 文档、字段、SDK、货架、号池、续租相关常见问题
- 针对签名失败、回调异常、错误码等问题,追问关键日志并给出排查路径
- 基于样例数据做单量趋势、成功率、失败率和异常波动分析
- 记录重复问题、误判和真实联调案例,形成后续 Skill 优化建议

不包括(留到后续迭代或明确不做):

- 自动生成 appsecret(必须人工审核)
- 自动改后台配置(必须走权限和审批)

- 自动上线(必须人工确认检查清单)
- 自动更新生产 Skill(必须先测试集上验证,再人工审核)

Part 8: 结语 · 业务 Agent 的目标

我们现在做 Agent,不是为了把它做成又一个聊天助手,而是要让它成为可以流程化、可控、可复盘的业务系统里的一部分。

业务 Agent 的目标不是替代所有人,而是:

- 把重复流程变成可复用能力
- 把个人经验变成组织资产
- 把大模型的表达能力放进可控的业务系统里

当一个业务 Agent 能稳定处理核心场景,且日志、Memory、Skill 更新机制跑通后,再抽象成通用底座;此时新业务只需替换 Workflow、Schema、Skill、知识库和验收集,不用重新发明整个 Agent 架构。

最后一句:Agent

架构里每个模块都有它存在的理由——不是为了炫技,而是为了让一个概率性的 LLM 能在确定性要求极高的企业业务流程里跑得稳、守得住边界、经得起复盘。产品经理的核心工作,就是把业务流程、字段规则、安全边界先定义清楚,然后让 LLM 在这些边界里发挥它理解表达的能力。

附录:模块速记

Workflow 管流程,Schema 管判断,Clarification Policy 管追问,Skill 管能力,Task State 管推进,Memory 管进化,Audit 和人工审核管风险,LLM 负责理解和表达增强。

版本信息

- 版本:v2.0(重构版)
- 日期:2026-06-11
- 基于:《企业业务 Agent 模块解析手册-增强版》(v1.0)重构
- 主要改动:新增「是什么+怎么工作+企业落地认知」三段前置,修正 Workflow 归位、图编号、模块按主链路重排
- 素材来源:wiki/concepts/(agent-definition / agent-loop / enterprise-agent / agent-paradigm-stages)、wiki/sources/(enterprise-agent-module-handbook /

handbook-page3-rewrite / pm-5min-hook / api-agent-template-breakdown)